

TECHNICAL REQUIREMENTS DOCUMENT (TRD)

Carbon Horizon

Learn Your Impact. Shape the Future.

Version: 2.0

1. Executive Summary

Carbon Horizon is an AI-powered Personal Sustainability Intelligence Platform that enables users to measure, understand, simulate, forecast, and reduce their carbon footprint through scientifically backed calculations, AI-driven sustainability guidance, and long-term behavioral tracking.

This document defines the technical architecture, system requirements, APIs, database standards, security requirements, deployment strategy, and implementation guidelines for the Carbon Horizon MVP.

2. Technical Objectives

The platform shall:

- Calculate user carbon footprints accurately
 - Identify major emission contributors
 - Generate AI-powered sustainability recommendations
 - Simulate future lifestyle scenarios
 - Forecast future emissions
 - Track sustainability goals
 - Monitor eco-friendly habits
 - Store historical user progress
 - Support cloud deployment and future scaling
-

3. System Architecture

High-Level Architecture

User ↓ React Frontend ↓ API Gateway Layer ↓ FastAPI Backend ↓ Business Services Layer |——
Authentication Service |—— Carbon Assessment Service |—— Carbon Calculation Engine |——
Simulation Engine |—— Forecast Engine |—— Habit Tracking Service |—— Goal Management Service

4. Architectural Principles

Principle 1

Business logic must reside in backend services.

No carbon calculations shall occur in frontend applications.

Principle 2

AI services must never generate carbon calculations.

AI shall only:

- Explain results
- Provide recommendations
- Generate action plans
- Answer sustainability questions

All numerical calculations must originate from backend services.

Principle 3

All carbon calculations must be traceable.

Each emission result must be linked to:

- Calculation version
 - Emission factor version
 - Data source
-

Principle 4

System must support future feature expansion without database redesign.

5. Technology Stack

Frontend

Framework:

React + TypeScript

UI:

Tailwind CSS

Charts:

Chart.js

State Management:

Redux Toolkit

Routing:

React Router

Validation:

React Hook Form + Zod

Backend

Framework:

FastAPI

Language:

Python 3.12+

Validation:

Pydantic

ORM:

SQLAlchemy

Migration Tool:

Alembic

Authentication:

JWT

Password Security:

bcrypt

Database

Development:

SQLite

Production:

PostgreSQL

Database Version:

PostgreSQL 16+

AI Services

Provider:

Gemini API

Purpose:

- Sustainability Guidance
 - Educational Responses
 - Action Planning
-

Cloud Infrastructure

Platform:

Google Cloud Platform

Services:

- Cloud Run
 - Cloud SQL
 - Secret Manager
 - Artifact Registry
 - Cloud Logging
 - Cloud Monitoring
-

6. Functional Modules

Module 1: Authentication Service

Responsibilities

- Registration
- Login
- Logout
- Session Validation
- Password Reset
- Account Management

Features

- JWT Authentication
- Refresh Tokens
- Password Hashing
- Email Verification (Future Ready)

Endpoints

POST /api/auth/register

POST /api/auth/login

POST /api/auth/logout

POST /api/auth/refresh

GET /api/auth/profile

PUT /api/auth/profile

DELETE /api/auth/account

Module 2: Carbon Assessment Service

Purpose

Collect and validate user lifestyle information.

Input Categories

Transportation

- Car
- Motorcycle
- Bus
- Train
- Flight
- Bicycle

Energy

- Electricity Usage
- AC Usage
- LPG Usage
- Solar Usage

Food

- Vegetarian
- Mixed
- Non-Vegetarian

Waste

- Recycling Frequency
- Plastic Usage

Household

- Number of Residents

Outputs

- Daily Emissions
 - Monthly Emissions
 - Annual Emissions
 - Emission Breakdown
 - Carbon Score
-

Module 3: Carbon Calculation Engine

Purpose

Convert activity data into carbon emissions.

Formula

Carbon Emission = Activity Data × Emission Factor

Example

Transport

100 km × 0.192 kg CO₂e/km

= 19.2 kg CO₂e

Responsibilities

- Emission Factor Lookup
 - Emission Calculation
 - Source Breakdown
 - Carbon Score Calculation
 - Version Tracking
-

Module 4: AI Sustainability Coach

Purpose

Provide personalized sustainability guidance.

Inputs

- User Profile
- Carbon Assessment

- Habit History
- User Goals
- Simulation Results

Outputs

- Recommendations
- Sustainability Education
- Action Plans
- Goal-Based Guidance

Restrictions

AI cannot:

- Generate emissions
 - Calculate reductions
 - Modify carbon scores
-

Module 5: Carbon Reduction Simulator

Purpose

Allow users to test lifestyle changes.

Supported Simulations

Transportation

- Car → Bus
- Car → Bicycle
- Motorcycle → Train

Energy

- Electricity Reduction
- Reduced AC Usage
- Solar Adoption

Food

- Mixed → Vegetarian
- Non-Veg → Mixed

Waste

- Improved Recycling

Outputs

- Projected Emissions
 - Carbon Saved
 - Reduction Percentage
 - Goal Progress Impact
-

Module 6: Lifestyle Impact Forecast Engine

Purpose

Project future emissions.

Inputs

- Current Emissions
- Habit History
- Goals
- Simulations

Forecast Scenarios

Scenario A

Current Lifestyle

Scenario B

Partial Improvements

Scenario C

Recommended Improvements

MVP Method

Rule-Based Projection Model

Future Emissions = Current Emissions – Habit Impact – Planned Improvements

Future Version

Machine Learning Forecasting

Module 7: Goal Management Service

Responsibilities

- Goal Creation
- Goal Tracking
- Progress Monitoring
- Goal Completion

Example Goal

Reduce annual emissions by 20%

Target Date: 31 December 2026

Module 8: Habit Tracking Service

Activities

- Public Transport
- Recycling
- Electricity Savings
- Water Conservation
- Plastic Reduction

Outputs

- Streaks
 - Weekly Reports
 - Monthly Reports
 - Carbon Savings
-

Module 9: Dashboard Service

Responsibilities

Display:

- Carbon Score
- Emission Breakdown
- Goal Progress
- Forecasts
- Habit Trends
- Historical Performance

7. Carbon Methodology

Sources

Carbon factors shall be derived from:

- IPCC
- EPA
- IEA
- Government Energy Statistics

Version Control

Every factor shall include:

- Source
- Publication Date
- Version Number

Auditability

All assessments must store:

- Calculation Version
- Factor Version
- Timestamp

8. Database Design

users

id (UUID)

full_name

email

password_hash

age_group

lifestyle_type

city

country

created_at

updated_at

deleted_at

carbon_assessments

id

user_id

transport_emission

energy_emission

food_emission

waste_emission

total_emission

carbon_score

calculation_version

factor_version

created_at

updated_at

emission_inputs

id

assessment_id

transport_mode

distance_km

electricity_kwh

ac_hours

lpg_usage

solar_usage

diet_type

recycling_score

plastic_usage_score

household_size

created_at

goals

id

user_id

goal_name

target_reduction

target_date

current_progress

status

created_at

updated_at

simulations

id

user_id

scenario_name

current_emission

projected_emission

reduction_percentage

estimated_saved

simulation_data

created_at

updated_at

forecasts

id

user_id

forecast_type

created_at

updated_at

forecast_points

id

forecast_id
month_offset
predicted_emission
created_at

habits

id
user_id
habit_type
completed
carbon_saved
activity_date
created_at

ai_conversations

id
conversation_id
user_id
role
message
created_at

carbon_factors

id

category

subcategory

factor_value

unit

source

version

updated_at

9. API Design

Authentication

POST /api/auth/register

POST /api/auth/login

POST /api/auth/logout

GET /api/auth/profile

PUT /api/auth/profile

DELETE /api/auth/account

Assessment

POST /api/assessment/create

GET /api/assessment/history

GET /api/assessment/{id}

Dashboard

GET /api/dashboard

Simulator

POST /api/simulator/run

POST /api/simulator/save

GET /api/simulator/history

Forecast

POST /api/forecast/generate

GET /api/forecast/history

GET /api/forecast/{id}

Goals

POST /api/goals

GET /api/goals

PUT /api/goals/{id}

DELETE /api/goals/{id}

Habits

POST /api/habits

GET /api/habits

PUT /api/habits/{id}

DELETE /api/habits/{id}

AI Coach

POST /api/coach/chat

GET /api/coach/history

10. API Standards

Success Response

HTTP 200

```
{ "success": true, "data": {} }
```

Validation Error

HTTP 400

```
{ "success": false, "message": "Invalid Input" }
```

Unauthorized

HTTP 401

```
{ "success": false, "message": "Unauthorized" }
```

Server Error

HTTP 500

```
{ "success": false, "message": "Internal Server Error" }
```

11. Security Requirements

Authentication

JWT Access Tokens

Refresh Tokens

Password Security

bcrypt

Minimum 8 Characters

Input Validation

Pydantic Validation

Request Sanitization

Secrets Management

Google Secret Manager

API Protection

Rate Limiting

Request Throttling

CORS Protection

Database Security

Parameterized Queries

ORM Protection

12. Performance Requirements

API Response Time: < 2 Seconds

Dashboard Load Time: < 3 Seconds

Database Query Time: < 500ms

Concurrent Users: 1000+

Forecast Generation: < 3 Seconds

AI Response: < 5 Seconds

13. Logging & Monitoring

Application Logs

Authentication Logs

API Request Logs

Error Logs

AI Usage Logs

Forecast Execution Logs

Cloud Monitoring

Cloud Logging

Error Alerting

Health Monitoring

14. Deployment Architecture

Developer ↓ GitHub ↓ GitHub Actions ↓ Docker Build ↓ Artifact Registry ↓ Cloud Run ↓ FastAPI Services
↓ Cloud SQL PostgreSQL

15. CI/CD Pipeline

1. Push Code
 2. Run Linting
 3. Run Unit Tests
 4. Run Integration Tests
 5. Build Docker Image
 6. Push Artifact
 7. Deploy to Cloud Run
 8. Execute Health Checks
 9. Publish Deployment
-

16. Testing Strategy

Unit Testing

Coverage:

- Carbon Calculation Engine
- Forecast Engine
- Goal Engine
- Simulator

Target Coverage: 90%+

API Testing

Coverage:

- Authentication
- CRUD Operations
- Validation

Target Coverage: 90%+

Integration Testing

Coverage:

- Gemini Integration
- Database Operations
- Cloud Services

Target Coverage: 80%+

Security Testing

Coverage:

- JWT Validation
 - Access Control
 - Rate Limiting
 - Injection Prevention
-

17. Scalability Strategy

Phase 1

Single Cloud Run Instance

Phase 2

Auto Scaling

Read Replicas

Caching Layer

Phase 3

Microservices Architecture

Event-Driven Processing

Advanced Forecast Engine

18. Future Technical Enhancements

- OCR Utility Bill Scanner
 - ML-Based Forecasting
 - Real-Time Carbon APIs
 - Mobile Applications
 - Multi-Language Support
 - Community Features
 - Gamification
 - Smart Device Integration
-

19. Technical Success Criteria

The platform shall be considered technically successful if:

- Carbon calculations are scientifically traceable
- AI recommendations are generated successfully
- Simulations produce measurable reductions
- Forecasts are generated accurately
- Goal tracking functions correctly
- Core APIs remain within performance targets
- Security requirements are satisfied
- Cloud deployment is successful
- CI/CD pipeline executes automatically
- All MVP modules pass testing requirements